

Backend Assignment - Toll Plazas Between Two Pincodes

Develop a **Spring Boot application** that provides an API to determine the toll plazas located between two Indian pincodes. The application should simulate a route between the source and destination pincodes and return details about toll plazas along the way.

Requirements

1. API Functionality:

- Accept source and destination pincodes as input
- Return a list of toll plazas on the route, including:
 - Toll plaza name
 - Latitude and longitude
 - Distance from the source pincode.
- Use caching for subsequent API call

2. Assumptions:

- The route between two pincodes can be found using google maps or any other external APIs.
- Toll Plaza List in CSV file - <https://drive.google.com/file/d/1AIBnma7N1nbvA5L7fFmCUC4QT-Wdt3e/view>

3. Design Constraints:

- Handle edge cases, such as invalid pincodes, no toll plazas on the route, or the same source and destination pincode.
 - Provide proper validations and error messages for incorrect input.
-

API Specification

Request

Endpoint:

`POST /api/v1/toll-plazas`

Request Body:

```
{  
  "sourcePincode": "110001",  
  "destinationPincode": "560001"
```

```
}
```

Response

Success Response:

```
{
  "route": {
    "sourcePincode": "110001",
    "destinationPincode": "560001",
    "distanceInKm": 2100
  },
  "tollPlazas": [
    {
      "name": "Toll Plaza 1",
      "latitude": 28.7041,
      "longitude": 77.1025,
      "distanceFromSource": 200
    },
    {
      "name": "Toll Plaza 2",
      "latitude": 19.076,
      "longitude": 72.8777,
      "distanceFromSource": 1400
    }
  ]
}
```

Error Response (Invalid Pincode):

```
{
  "error": "Invalid source or destination pincode"
}
```

Error Response (No Toll Plazas):

```
{
  "route": {
    "sourcePincode": "110001",
    "destinationPincode": "560001",
    "distanceInKm": 2100
  },
  "tollPlazas": []
}
```

Error Response (Same Source and Destination):

```
{
  "error": "Source and destination pincodes cannot be the same"
}
```

}

Evaluation Criteria

- Assignment code submitted to be in **Java**
- REST Compliant APIs, (No UI)
- Testable by Postman, (No UI)
- Will prefer classes properly refactored and following design patterns,
- Clarity of API documentation and how to run the project.
- Will Prefer TDD, JUnit , **Test Coverage**: Comprehensive test cases covering various scenarios.
- Efficient database operations (e.g., upsert for repository data).
- Code quality and structure (e.g., modular design, clear separation of concerns).
- **Error Handling**: Robust validation and error messages. Also Graceful handling of invalid or edge-case scenarios.

Testing your application

Try with Mappls - Your output should be 80%-90% of same tolls

<https://www.mappls.com/direction-from-560064.-bengaluru.-karnataka?-to-411045.-pune.-m-aharashtra>

